AI & Machine Learning Overview - Dermascan AI

1. Introduction

This component of the project focuses on the automated detection of skin diseases using Deep Learning. We designed and trained a Convolutional Neural Network (CNN) capable of classifying skin conditions into 8 distinct categories.

2. Model Architecture

We utilized **Transfer Learning** to leverage the power of pre-trained networks, ensuring high accuracy even with a limited dataset.

Base Model: MobileNetV2

* **Selection:** We chose MobileNetV2 because it is lightweight and efficient, making it ideal for web deployment where response time is critical.
* **Pre-training:** The model was pre-trained on the **ImageNet** dataset, allowing it to recognize fundamental visual features (edges, textures) right out of the box.
* **Input Shape:** The network accepts images of size **224x224 pixels** with 3 color channels (RGB).

Custom Classification Head

To adapt MobileNetV2 for our specific task of skin disease detection, we added custom layers on top of the frozen base:

1. **Global Average Pooling:** Reduces the spatial dimensions of the feature map.
2. **Dropout (0.3):** Randomly deactivates 30% of neurons during training to prevent overfitting.
3. **Dense Layer (256 units):** A fully connected layer with **ReLU** activation to learn complex non-linear patterns.
4. **Batch Normalization:** Stabilizes the learning process and accelerates training.
5. **Dropout (0.3):** A second dropout layer for added regularization.
6. **Output Layer (8 units):** A final dense layer with **Softmax** activation to output probabilities for the 8 disease classes.

3. Data Processing & Augmentation

To improve the model's robustness, we implemented a data augmentation pipeline using TensorFlow's `ImageDataGenerator`. This artificially expands the dataset by creating modified versions of images during training.

* **Rescaling:** Initial pixel values (0-255) are normalized to the 0-1 range.
* **Rotation:** Random rotations up to 20 degrees.
* **Shifting:** Random horizontal and vertical shifts (10%).
* **Shearing & Zooming:** Random shear and zoom transformations (10%).
* **Flipping:** Random horizontal flips to handle orientation invariance.

4. Training Configuration

The model was compiled and trained with the following hyperparameters:

* **Optimizer: Adam** (Adaptive Moment Estimation) for efficient gradient descent.
* **Loss Function: Categorical Crossentropy**, suitable for multi-class classification.
* **Metrics: Accuracy**.
* **Batch Size:** 32.
* **Epochs:** 30.

Callbacks used for Optimization:

* **ModelCheckpoint:** Saves only the best-performing model based on validation accuracy.
* **EarlyStopping:** Stops training if the model stops improving for 5 epochs to save time.
* **ReduceLROnPlateau:** Reduces the learning rate if the validation loss stagnates, allowing the model to fine-tune its weights.

5. Technology Stack

* **Framework:** TensorFlow & Keras
* **Language:** Python 3.12
* **Format:** The final model is saved in `.h5` format (`skin_cnn_mobilenetv2.h5`) for integration with the Django backend.